global replacements in the content generated by the controller's templates (for example, by substituting a customer's name for the string `<customer/>` in the response body). Another use might be compressing the response if the user's browser supports it.

Around callbacks wrap the execution of actions. You can write an around callback in two different styles. In the first, the callback is a single chunk of code. That code is called before the action is executed. If the callback code invokes `yield`, the action is executed. When the action completes, the callback code continues executing.

Thus, the code before the `yield` is like a before action callback, and the code after is the after action callback. If the callback code never invokes `yield`, the action isn't run—this way you can achieve the same result as a before action callback returning `false`.

The benefit of around callbacks is that they can retain context across the invocation of the action.

As well as passing `around_action` the name of a method, you can pass it a block or a filter class.

If you use a block as a callback, it'll be passed two parameters: the controller object and a proxy for the action. Use `call` on this second parameter to invoke the original action.

A second form allows you to pass an object as a callback. This object should implement a method called `filter`. This method will be passed the controller object. It yields to invoke the action.

Like before and after callbacks, around callbacks take `:only` and `:except` parameters.

Around callbacks are (by default) added to the callback chain differently: the first around action callback added executes first. Subsequently added around callbacks will be nested within existing around callbacks.